Flood Intelligence System

Repository File Guide & Extension Notes

Use this document as the quick orientation guide after cloning or forking the repository. It explains which files are core, which are deployment-specific, and where to make changes safely.

1. Repository at a glance

File / data	Summary
digital_nz_api.py	Collection script. Queries DigitalNZ for event/flood terms, normalises records and upserts them into `news_articles`.
api.py	Small DigitalNZ client used by the collector; builds requests and parses API results.
flood_event_verified_pipeline.py	Main research pipeline. Builds candidates, verifies article-event membership, resolves primary assignments, performs grouped Event -> Incident -> Impact extraction and imports results.
table_schema.sql	Creates the structured PostgreSQL tables and includes useful verification/inspection queries.
commands.txt	Short command reference for batch preparation, submission, status, download and import.
final_dashboard_verified_updated.py	PostgreSQL-backed Dash dashboard for development/research use.
final_dashboard_csv.py	CSV-backed Dash dashboard for lightweight/public deployment.
api/index.py / index.py	Vercel adapter: adds repository root to Python path, imports the Dash app, and exposes `app.server`.
vercel.json	Routes requests to the Vercel Python entrypoint. Repository configuration should use the exact filename `vercel.json`.
requirements.txt	Dash, pandas, Plotly, OpenAI, dotenv and deployment dependencies.

2. Data files

CSV	What it represents
news_articles.csv	4,742 collected source articles and metadata/content.
flood_article_event_validation.csv	9,268 article-event decisions including confidence, temporal/location/context matches and primary assignment.
flood_event_articles.csv	1,029 verified articles retained for final events.
flood_events.csv	34 structured events with summaries, date ranges, regions and evidence.
flood_incidents.csv	176 event-linked incidents with location/date/evidence.
flood_impacts.csv	358 incident/event-linked impacts with category, type, value, location and evidence.

3. How to work with the codebase

Collect more articles

Extend `SEARCH_QUERIES` or add another collector that writes the same `news_articles` fields. Keep URL uniqueness/deduplication intact.

Improve event classification

Change candidate filtering or the verification prompt/model inside `flood_event_verified_pipeline.py`. Preserve the validation audit table so experiments remain traceable.

Change incident/impact taxonomy

Update the extraction instructions and database/schema expectations together. Avoid changing only the UI labels.

Test another LLM

Keep the same input sample, prompt and output schema first. Swap only the model/provider so model comparisons remain scientifically interpretable.

Improve mapping

Add precise coordinates or authoritative geocoding. Keep centroid fallbacks clearly labelled as approximate.

Extend the dashboard

Add new filters/charts against the structured CSV/DB fields. For production, modify `final_dashboard_csv.py`; for database research, modify `final_dashboard_verified_updated.py`.

4. Minimal local setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

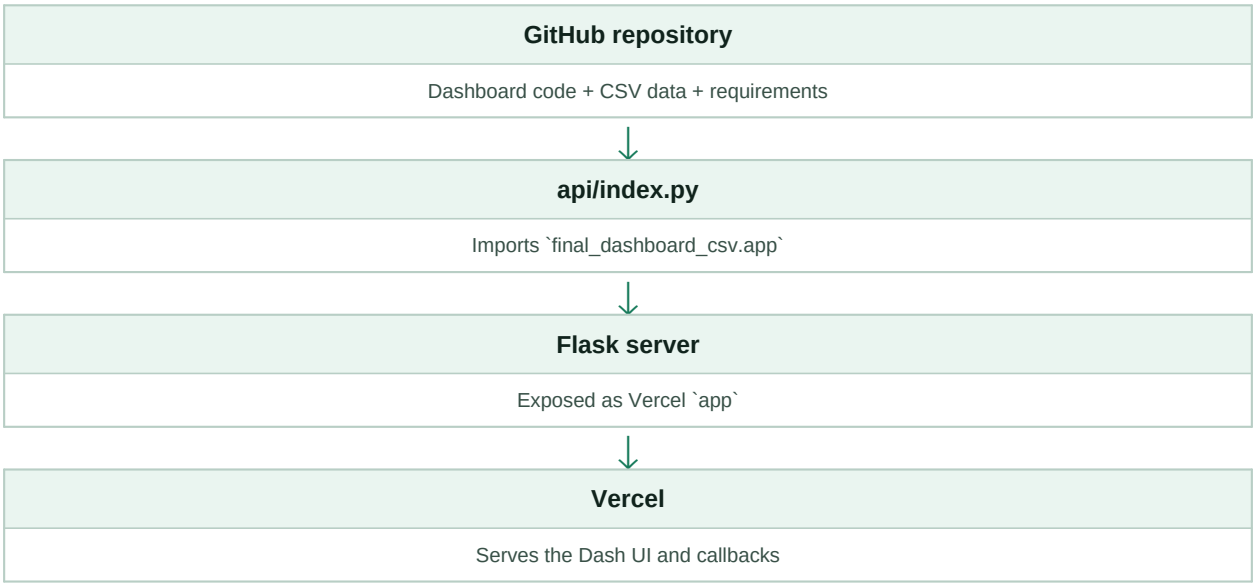
Create a local `.env` containing the required API/database variables. Never commit `.env`, API keys or database passwords.

For the CSV dashboard, keep the four dashboard CSVs in the same directory as `final_dashboard_csv.py`, then run:

```
python final_dashboard_csv.py
```

5. Vercel deployment path

The Vercel adapter imports the CSV Dash app and exposes the underlying Flask server. The routing configuration forwards application requests to that entrypoint.



Deployment note: Vercel expects the configuration filename `vercel.json`. If the repository currently contains `versel.json`, rename it. Also keep `OPENAI_API_KEY` and `OPENAI_MODEL` in Vercel Environment Variables rather than GitHub.

6. Recommended fork workflow

- Fork/clone the repository and create a feature branch.
- Create `.env`; never reuse committed credentials.
- Run the CSV dashboard first to confirm the UI works without database dependencies.
- Use a small validation subset before running costly full-corpus LLM jobs.
- Keep generated batch outputs and experimental artefacts separate from source code.
- Document changes to prompts, thresholds, models and taxonomy so results remain reproducible.
- Before merging, test both the pipeline output schema and dashboard loading.

7. What not to assume

CSV exports are snapshots, not the authoritative processing logic. The verification confidence is not a statistically calibrated probability. News-derived impacts are evidence from reporting, not a complete record of real-world impacts. The dashboard is for research/exploration and should not be presented as an operational warning service.